

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant est capable de :

- Manipuler les listes (création, ajout, suppression, parcours)
- Utiliser les méthodes de liste : `append()`, `remove()`, `sort()`, `index()`
- Comprendre l'indicage et le slicing
- Implémenter des algorithmes de recherche et de tri simples
- Distinguer liste et tuple
- Utiliser les compréhensions de listes pour filtrer des données

Plan du TP

Phase	Durée	Activité
Accueil et révision	5 min	Rappel sur les listes
Exercice 1	15 min	Manipulation de listes
Exercice 2	15 min	Recherche dans une liste
Exercice 3	20 min	Liste de notes avec coefficients
Exercice 4	20 min	Tri à bulles
Exercice 5	15 min	Filtrage d'une liste
QCM et bilan	15 min	Auto-évaluation et conclusion

Exercice 1 — Manipulation de listes (15 min)

Énoncé : Créer un script `listes.py` qui :

1. Crée une liste vide `notes`
2. Ajoute 5 notes saisies par l'utilisateur
3. Affiche la liste
4. Affiche la note max, min et la moyenne
5. Supprime la note la plus basse
6. Affiche la liste triée

Exemple attendu :

```
Note 1 : 12
Note 2 : 15
```

```

Note 3 : 8
Note 4 : 19
Note 5 : 10
Liste : [12.0, 15.0, 8.0, 19.0, 10.0]
Max : 19.0 – Min : 8.0 – Moyenne : 12.80
Après suppression de la plus basse : [10.0, 12.0, 15.0, 19.0]

```

Corrigé :

```

notes = []
for i in range(5):
    n = float(input(f"Note {i+1} : "))
    notes.append(n)

print(f"Liste : {notes}")
print(f"Max : {max(notes)} – Min : {min(notes)} – Moyenne : {sum(notes)/len(notes):.2f}")

notes.remove(min(notes))
notes.sort()
print(f"Après suppression de la plus basse : {notes}")

```

Exercice 2 — Recherche dans une liste (15 min)

Énoncé : Créer un script recherche.py qui :

1. Crée une liste de prénoms
2. Demande un prénom à l'utilisateur
3. Indique si le prénom est dans la liste et à quelle position
4. Propose d'ajouter le prénom s'il n'existe pas

Exemple attendu :

```

Liste actuelle : ['Ali', 'Sara', 'Mohamed', 'Leila', 'Ahmed']
Prénom à chercher : Sara
Sara trouvé à la position 2.

```

```

flowchart TD
    A[Début] --> B[Saisir prénom]
    B --> C{Prénom dans liste ?}
    C -->|Oui| D[Afficher position]
    C -->|Non| E[Ajouter ?]
    E -->|Oui| F[Ajouter et trier]
    E -->|Non| G[Fin]
    D --> G
    F --> G

```

Corrigé :

```

prenoms = ["Ali", "Sara", "Mohamed", "Leila", "Ahmed"]
print(f"Liste actuelle : {prenoms}")

recherche = input("Prénom à chercher : ").capitalize()

if recherche in prenoms:
    pos = prenoms.index(recherche) + 1
    print(f"{recherche} trouvé à la position {pos}.")
else:
    print(f"{recherche} n'est pas dans la liste.")
    ajout = input("Voulez-vous l'ajouter ? (o/n) : ")
    if ajout == "o":
        prenoms.append(recherche)

```

```
prenoms.sort()
print(f"Nouvelle liste : {prenoms}")
```

Exercice 3 — Liste de notes avec coefficients (20 min)

Énoncé : Créer un script `notes_coef.py` qui :

1. Demande le nombre de matières
2. Pour chaque matière, demande le nom, la note et le coefficient
3. Stocke les données dans une liste de dictionnaires
4. Calcule la moyenne pondérée
5. Affiche un tableau récapitulatif

Exemple attendu :

```
Nombre de matières : 3

--- Matière 1 ---
Nom : Maths
Note (0-20) : 14
Coefficient : 4

--- Matière 2 ---
Nom : Français
Note (0-20) : 12
Coefficient : 3

--- Matière 3 ---
Nom : Anglais
Note (0-20) : 16
Coefficient : 2

Moyenne pondérée : 13.78/20
```

Corrigé :

```
matieres = []
nb = int(input("Nombre de matières : "))

for i in range(nb):
    print(f"\n--- Matière {i+1} ---")
    nom = input("Nom : ")
    note = float(input("Note (0-20) : "))
    coef = float(input("Coefficient : "))
    matieres.append({"nom": nom, "note": note, "coef": coef})

total_points = sum(m["note"] * m["coef"] for m in matieres)
total_coefs = sum(m["coef"] for m in matieres)
moyenne = total_points / total_coefs if total_coefs > 0 else 0

print("\n" + "="*45)
print(f"{'Matière':<15} {'Note':<8} {'Coef':<8} {'Total':<8}")
print("="*45)
for m in matieres:
    total = m["note"] * m["coef"]
    print(f"{'nom':<15} {'note':<8.2f} {'coef':<8.2f} {'total':<8.2f}")
print("="*45)
print(f"Moyenne pondérée : {moyenne:.2f}/20")
```

Exercice 4 — Tri à bulles (20 min)

Énoncé : Créer un script `tri_bulles.py` qui implémente l'algorithme de tri à bulles sans utiliser `sort()`. Le programme doit :

1. Créer une liste aléatoire de 10 nombres
2. Afficher la liste originale
3. Trier avec l'algorithme à bulles
4. Afficher la liste triée
5. Compter le nombre d'échanges effectués

```

flowchart TD
    A[Liste non triée] --> B[i de 0 à n-1 ]
    B --> C[j de 0 à n-i-2 ]
    C --> D{ t[j] > t[j+1] ? }
    D -->|Oui| E[Échanger t[j] ↔ t[j+1]]
    D -->|Non| F[Passer]
    E --> G[j suivant ]
    F --> G
    G --> H{ Fin parcours ? }
    H -->|Non| C
    H -->|Oui| I[i suivant ]
    I --> J{ Liste triée ? }
    J -->|Non| B
    J -->|Oui| K[Liste triée]

```

Corrigé :

```

import random

def tri_bulles(liste):
    """Tri à bulles, retourne le nombre d'échanges."""
    n = len(liste)
    echanges = 0
    for i in range(n):
        for j in range(0, n - i - 1):
            if liste[j] > liste[j + 1]:
                liste[j], liste[j + 1] = liste[j + 1], liste[j]
                echanges += 1
    return echanges

liste = [random.randint(1, 100) for _ in range(10)]
print(f"Originale : {liste}")

echanges = tri_bulles(liste)
print(f"Triée : {liste}")
print(f"Échanges : {echanges}")

```

Exercice 5 — Filtrer une liste (15 min)

Énoncé : Créer un script `filtre.py` qui :

1. Crée une liste de 20 nombres aléatoires entre 0 et 100
2. Crée une nouvelle liste contenant uniquement les nombres pairs
3. Crée une nouvelle liste contenant uniquement les nombres ≥ 50
4. Affiche les trois listes avec leurs longueurs

Exemple attendu :

Tous : [12, 45, 78, 23, 56, 91, 34, 67, 8, 90, ...] (20 éléments)
 Pairs : [12, 78, 56, 34, 8, 90, ...] (11 éléments)
 ≥ 50 : [78, 56, 91, 67, 90, ...] (8 éléments)

Corrigé :

```
import random

nombres = [random.randint(0, 100) for _ in range(20)]
pairs = [n for n in nombres if n % 2 == 0]
grands = [n for n in nombres if n >= 50]

print(f"Tous : {nombres} ({len(nombres)} éléments)")
print(f"Pairs : {pairs} ({len(pairs)} éléments)")
print(f"≥ 50 : {grands} ({len(grands)} éléments)")
```

QCM d'auto-évaluation

- Quelle méthode permet d'ajouter un élément à la fin d'une liste ?
 - `insert()` ✓
 - `append()`
 - `add()`
- Quelle est la complexité du tri à bulles dans le pire des cas ?
 - $O(n)$
 - $O(n^2)$ ✓
 - $O(\log n)$
- Quel index correspond au premier élément d'une liste Python ?
 - 1
 - 0 ✓
 - 1
- Comment supprimer la première occurrence d'une valeur dans une liste ?
 - `del()`
 - `remove()` ✓
 - `pop()`
- Que retourne `len([1, 2, 3])` ?
 - 2
 - 3 ✓
 - 4

Grille d'évaluation TP 05

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Création et manipulation de listes	20 %	Ne sait pas créer de liste	Opérations basiques	Maîtrise complète
Méthodes de liste	20 %	N'utilise aucune méthode	Quelques méthodes	Toutes les méthodes

Algorithmes sur listes	20 %	Aucun algorithme	Algorithme partiel	Algorithme correct
Compréhension de listes	20 %	Ne connaît pas	Utilisation hasardeuse	Utilisation appropriée
Dictionnaires imbriqués	20 %	Ne maîtrise pas	Utilisation partielle	Utilisation correcte

Déroulement séance — 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 — Manipulation listes	15 min	20 min
Exercice 2 — Recherche	15 min	35 min
Exercice 3 — Notes avec coeff	20 min	55 min
Exercice 4 — Tri à bulles	20 min	75 min
Exercice 5 — Filtrage	15 min	90 min
Total	90 min	

Note pédagogique ENS

Ce TP introduit les listes Python comme structure de données fondamentale. Le tri à bulles (exercice 4) est l'occasion d'illustrer concrètement la complexité algorithmique $O(n^2)$ vue en cours théorique. En situation de classe au secondaire, on peut :

- Collège (cycle 4) : Utiliser l'exercice 2 (recherche) pour apprendre à organiser des données. Demander aux élèves de chercher un mot dans le dictionnaire papier, puis de comparer avec l'algorithme informatique.
- Lycée (SNT/NSI) : L'exercice 3 (notes avec coefficients) est un outil de suivi scolaire réel. Les futurs enseignants peuvent le reproduire avec leurs propres classes pour automatiser le calcul des moyennes.
- Visualisation pédagogique : Avant de coder le tri à bulles, utiliser des jetons de couleurs au tableau pour simuler les échanges. Cela permet de comprendre le mécanisme sans se perdre dans la syntaxe.
- Prolongement : Proposer aux étudiants de comparer le tri à bulles avec `list.sort()` (TimSort, $O(n \log n)$) sur une grande liste (10 000 éléments) pour mesurer la différence de performance.

Les compréhensions de listes (exercice 5) sont une spécificité Python qu'il est important de maîtriser pour écrire du code lisible et concis — un atout pour tout enseignant qui veut montrer des exemples clairs à ses élèves.